TGui — Documentation

Interface graphique pour Tarantool

Table des matières

Référence des fonctionnalités — TGui **2**

Table des matières . . . 2

1. Espaces (tables) . . . 3
    - Opérations UI . . . 3
    - API GraphQL . . . 3

Requêtes imbriquées (Nesting) . . . 4
    - Back-references avec pagination et filtrage . . . 4
    - Accès direct aux enregistrements d'un espace via `Space.records` . . . 4

2. Champs . . . 5
    - Types disponibles . . . 5
    - Champ calculé (colonne calculée) . . . 5
    - Trigger formula . . . 5
    - Langages de formule . . . 6
    - Helper `space` — accès aux autres espaces . . . 6
    - Réordonnancement . . . 6
    - Représentation personnalisée des relations (`reprFormula`) . . . 6
    - Conversion de type de champ . . . 7
    - API GraphQL . . . 7

3. Relations . . . 9
    - API GraphQL . . . 9
    - Formule de représentation . . . 10

4. Données (enregistrements) . . . 10
    - Édition dans la grille . . . 10
    - Filtrage . . . 11
    - Opérateurs de filtre . . . 11
    - Filtres formule et traversée FK . . . 11
    - API GraphQL . . . 12

5. Vues classiques . . . 13
    - API GraphQL . . . 13

6. Vues personnalisées (YAML) . . . 14
    - Éditeur . . . 14
    - Structure de base . . . 14
    - Widget . . . 14
    - Dépendances (filtrage inter-widgets) . . . 15
    - Facteurs de proportion . . . 15
    - Colonnes personnalisées . . . 15
    - Exemple complet . . . 15

Widget agrégat . . . 16
API GraphQL . . . 17
7. Authentification . . . 18
Flux de connexion . . . 18
Déconnexion . . . 18
Utilisateur courant . . . 18
Changer son propre mot de passe . . . 18
8. Utilisateurs et groupes . . . 19
Utilisateurs . . . 19
Groupes . . . 19
9. Droits (permissions) . . . 20
10. Export et import de snapshots . . . 20
Export . . . 20
Import . . . 23
Modes d'import . . . 23
API GraphQL correspondante . . . 23
11. API GraphQL complète . . . 24
Queries . . . 24
Mutations . . . 24
Architecture technique . . . 25

Référence des fonctionnalités — TGui

TGui est inspiré de Grist — pour tout usage sérieux ou en production, l'utilisateur est invité à se tourner vers ce dernier en priorité.

TGui expose toutes ses fonctionnalités via une **API GraphQL** accessible sur `http://localhost:8080/graphql`. L'interface web est un client de cette même API ; tout ce qui est faisable dans l'UI l'est aussi directement via des requêtes GraphQL.

Table des matières

1. Espaces (tables)
2. Champs
3. Relations
4. Données (enregistrements)
5. Vues classiques
6. Vues personnalisées (YAML)
7. Authentification
8. Utilisateurs et groupes
9. Droits (permissions)
10. Export et import de snapshots
11. API GraphQL complète

---

1. Espaces (tables)

Un **espace** est une table de données définie par l'utilisateur. Il contient des **champs** (colonnes) et des **enregistrements** (lignes).

Opérations UI

Action	Comment
Créer	Clic sur + dans la section Données
Renommer	Bouton **(crayon)** dans la barre d'outils de l'espace
Supprimer	Bouton **[suppr]** (poubelle à côté du titre)
Naviguer	Clic sur le nom dans la barre latérale

API GraphQL

```
# Lister tous les espaces
query {
  spaces { id name description fields { id name fieldType } }
}

# Détail d'un espace, y compris les enregistrements (imbriqués)
query {
  space(id: "...") {
    id name fields { id name fieldType formula }
    records(limit: 10, offset: 0, filter: { field: "status", op: EQ, value:
 ↪  "active" }) {
      items { id data }
      total
    }
  }
}

# Créer
mutation {
  createSpace(input: { name: "livres", description: "Catalogue des titres" })
 ↪  { id }
}

# Renommer
mutation {
  updateSpace(id: "...", input: { name: "nouveau_nom" }) { id name }
}
```

```
# Supprimer
mutation { deleteSpace(id: "...") }
```

Requêtes imbriquées (Nesting)

En plus de pouvoir récupérer les relations simples (FK) directement dans les enregistrements, il est possible de **requêter des collections d'enregistrements liés (back-references) avec pagination et filtrage directement au sein d'une requête parente**.

Back-references avec pagination et filtrage

Les champs de back-reference générés dynamiquement (basés sur les relations inverses) supportent les arguments `limit`, `offset` et `filter` :

```
# Récupérer les utilisateurs et leurs tâches, en filtrant les tâches par titre
query {
  users {
    id
    username
    groups {
      id
      name
    }
  }
}
```

Accès direct aux enregistrements d'un espace via `Space.records`

Le type `Space` expose un champ `records` qui permet de récupérer directement les enregistrements d'un espace donné, avec pagination et filtrage, sans passer par la racine `Query.records` :

```
# Récupérer les détails d'un espace et ses 10 premiers enregistrements
query {
  space(id: "mon_espace_id") {
    name
    records(limit: 10, offset: 0) {
      items { id data }
      total
    }
  }
}
```

---

2. Champs

Un **champ** est une colonne d'un espace. Chaque champ a un type, un nom, et peut porter une formule.

Types disponibles

Type	Description
`String`	Chaîne de caractères
`Int`	Entier
`Float`	Nombre décimal
`Boolean`	Booléen (`true` / `false`)
`UUID`	UUID généré automatiquement
`Sequence`	Entier auto-incrémenté (clé primaire naturelle)
`Any`	Type libre (JSON)
`Map`	Objet JSON
`Array`	Tableau JSON
`Datetime`	Date et heure
`Relation`	Référence vers un enregistrement d'un autre espace (affichage personnalisable via `reprFormula`)

Champ calculé (colonne calculée)

Un champ calculé évalue une **expression MoonScript** à chaque lecture. La valeur n'est **pas stockée**.

La formule est le corps d'une fonction `(self, space) -> <formule>`. En MoonScript, `@champ` est un raccourci pour `self.champ` :

Exemple : **`nom_complet`** dans **`auteurs`** — combine prénom, particule et nom en tenant compte des valeurs nulles :

```
parts = {}
table.insert(parts, @prenom) if @prenom and @prenom != ""
table.insert(parts, @particule) if @particule and @particule != ""
table.insert(parts, @nom) if @nom and @nom != ""
table.concat(parts, " ")
```

Trigger formula

Une trigger formula est évaluée et son résultat **stocké** lors de chaque création ou modification de l'enregistrement (ou uniquement lors du changement de champs listés dans `triggerFields`).

Exemple : **`cote_auto` dans `livres`** — génère une cote bibliographique automatique à partir des trois premières lettres du titre et de l'année (déclenché sur `titre` et `annee`) :

```
-- triggerFields: ["titre", "annee"]
prefix = (@titre or "")\upper!\sub(1, 3)\gsub("[^%A]", "")
annee  = if @annee then tostring(@annee) else "????"
"#{prefix}-#{annee}"
```

Langages de formule

Valeur	Description
`moonscript`	MoonScript (compilé ⟶ Lua à la volée)
`lua`	Lua natif

Helper `space` — accès aux autres espaces

Les formules et triggers reçoivent un deuxième paramètre `space` qui permet d'accéder à tous les enregistrements d'un autre espace (full scan) :

```
-- Compter les exemplaires disponibles pour ce livre
n = 0
for e in *space("exemplaires")
  n += 1 if e.livre_id == @id and e.disponible
n

-- Récupérer le libellé du genre d'un livre
next(g for g in *space("genres") when g._id == @genre_id)?.libelle
```

`space("nom")` retourne une liste Lua de tous les enregistrements de l'espace `nom`, chaque enregistrement étant un objet avec `_id` (identifiant), plus tous les champs de données. Retourne `{}` si l'espace n'existe pas.

> **Attention** : le full scan est adapté aux espaces de petite taille (quelques milliers d'enregistrements). Pour des volumes importants, préférer les relations et les widgets agrégats.

Réordonnancement

Les champs peuvent être réordonnés par glisser-déposer (::) dans le panel Champs.

Représentation personnalisée des relations (`reprFormula`)

Pour les champs de type `Relation`, `reprFormula` permet de contrôler comment la relation est affichée dans l'interface (grille, formulaires, etc.). La formule reçoit l'enregistrement lié en paramètre (`self`) et doit retourner une chaîne de caractères.

Exemple : Pour un champ `auteur_id` qui pointe vers l'espace `auteurs` :

```
# Afficher "Nom Prénom" au lieu de l'UUID
"#{@nom} #{@prenom}"
```

```lua
-- Version Lua équivalente
return self.nom .. " " .. self.prenom
```

Conversion de type de champ

La mutation `changeFieldType` permet de convertir un champ d'un type vers un autre, avec une formule de conversion optionnelle pour transformer les données existantes.

Exemple : Convertir un champ `String` en `Int` :

```
mutation {
  changeFieldType(fieldId: "...", input: {
    fieldType: Int,
    conversionFormula: "tonumber(self.amount)",
    language: "lua"
  }) { id name fieldType }
}
```

Si `conversionFormula` est omise, les valeurs sont converties automatiquement quand c'est possible, ou mises à `null` sinon.

API GraphQL

```
# Ajouter un champ simple
mutation {
  addField(spaceId: "...", input: {
    name: "age", fieldType: Int, notNull: false
  }) { id name }
}

# Ajouter une colonne calculée
mutation {
  addField(spaceId: "...", input: {
    name: "nom_complet", fieldType: String,
    formula: "parts = {}\ntable.insert(parts, @prenom) if @prenom and @prenom
 ↪  != \"\"\ntable.insert(parts, @particule) if @particule and @particule !=
 ↪  \"\"\ntable.insert(parts, @nom) if @nom and @nom !=
 ↪  \"\"\ntable.concat(parts, \" \")",
    language: "moonscript"
  }) { id }
}

# Ajouter une trigger formula
```

```
# (se déclenche sur changement de "titre" ou "annee")
mutation {
  addField(spaceId: "...", input: {
    name: "cote_auto", fieldType: String,
    formula: "prefix = (@titre or \"\")\\upper!\\sub(1, 3)\\gsub(\"[^%A]\",
↪ \"\")\nannee  = if @annee then tostring(@annee) else
↪ \"????\"\n\"#{prefix}-#{annee}\"",
    triggerFields: ["titre", "annee"],
    language: "moonscript"
  }) { id }
}

# Modifier un champ existant
mutation {
  updateField(fieldId: "...", input: {
    formula: "self.prenom .. \" \" .. self.nom",
    language: "lua"
  }) { id name formula }
}

# Supprimer
mutation { removeField(fieldId: "...") }

# Réordonner
mutation {
  reorderFields(
    spaceId: "...",
    fieldIds: ["id1", "id2", "id3"]
  ) { id position }
}

# Changer le type d'un champ
mutation {
  changeFieldType(fieldId: "...", input: {
    fieldType: Int,
    conversionFormula: "tonumber(self.amount)",
    language: "lua"
  }) { id name fieldType }
}
```

3. Relations

Une **relation** lie un champ d'un espace (champ de type `Relation`) à un autre espace. Elle permet d'activer le filtrage dépendant dans les vues personnalisées.

Les relations **récursives** (vers l'espace lui-même) sont supportées, pour modéliser des structures arborescentes (ex. catégories, généalogies).

API GraphQL

```graphql
# Lister les relations d'un espace
query {
  relations(spaceId: "...") {
    id name fromFieldId toSpaceId toFieldId reprFormula
  }
}

# Créer une relation livres → auteurs
mutation {
  createRelation(input: {
    name: "livre_auteur",
    fromSpaceId: "livres-id",
    fromFieldId: "auteur_id-field-id",
    toSpaceId:   "auteurs-id",
    toFieldId:   "id-field-id"
  }) { id }
}

# Créer une relation livres → genres
mutation {
  createRelation(input: {
    name: "livre_genre",
    fromSpaceId: "livres-id",
    fromFieldId: "genre_id-field-id",
    toSpaceId:   "genres-id",
    toFieldId:   "id-field-id"
  }) { id }
}

# Créer une relation exemplaires → livres
mutation {
  createRelation(input: {
    name: "exemplaire_livre",
    fromSpaceId: "exemplaires-id",
    fromFieldId: "livre_id-field-id",
    toSpaceId:   "livres-id",
```

```
    toFieldId:   "id-field-id"
  }) { id }
}

# Supprimer
mutation { deleteRelation(id: "...") }

# Définir la formule de représentation d'une relation livres → auteurs
mutation {
  updateRelation(id: "...", input: {
    reprFormula: "(@particule and @particule .. ' ' or '') .. @nom .. (@prenom
 ↪  and ' ' .. @prenom or '')"
  }) {
    id reprFormula
  }
}
```

Formule de représentation

Une relation peut avoir un champ **`reprFormula`** : une expression MoonScript évaluée sur chaque enregistrement cible pour produire un libellé lisible.

- La variable `self` (accessible via `@champ`) est liée à l'enregistrement cible.
- Exemples : `@libelle`, `@nom .. ' ' .. @prenom`
- Les champs FK sont résolus automatiquement : `@genre_id.libelle` suit la relation `genre_id` et retourne le champ `libelle` de l'enregistrement genres correspondant.
- Le chaînage est possible sur plusieurs niveaux : `@livre_id.genre_id.libelle` (exemplaire → livre → genre).
- Quand elle est définie, les colonnes FK de la grille affichent le résultat de la formule au lieu de l'identifiant brut.
- En mode édition, une liste déroulante présente tous les enregistrements cibles avec leur libellé calculé.
- Si aucune formule n'est définie, la colonne utilise le champ `_repr` de l'espace cible (s'il existe), sinon l'identifiant brut.

---

4. Données (enregistrements)

Édition dans la grille

- Clic sur une cellule pour entrer en mode édition.
- **Entrée** ou **Tab** pour valider et passer à la suivante.
- Ligne vide en bas : saisir pour créer un nouvel enregistrement.
- Cocher une ou plusieurs lignes puis cliquer **[suppr]** pour supprimer.

Filtrage

L'API supporte un filtrage composable :

```
query {
  records(
    spaceId: "..."
    filter: {
      or: [
        { field: "titre", op: STARTS_WITH, value: "Le" }
        { field: "annee", op: GTE, value: "2000" }
      ]
    }
    limit: 50
    offset: 0
  ) {
    items { id data }
    total
  }
}
```

Opérateurs de filtre

Opérateur	Signification
`EQ`	Égal à
`NEQ`	Différent de
`LT` / `GT`	Inférieur / supérieur (strict)
`LTE` / `GTE`	Inférieur / supérieur ou égal
`CONTAINS`	Contient la sous-chaîne
`STARTS_WITH`	Commence par

Filtres formule et traversée FK

En plus des filtres par champ et opérateur, il est possible d'exprimer des conditions arbitraires en **MoonScript** via `formula`. Ces formules ont accès au **proxy FK** : les champs de type relation sont résolus automatiquement vers l'enregistrement lié, et le chaînage sur plusieurs niveaux est supporté.

```
# Filtrer les livres dont le genre est 'Roman' (via traversée FK)
query {
  records(
    spaceId: "livres-id"
```

```
    filter: { formula: "@genre_id.libelle == 'Roman'", language: "moonscript"
 ↪  }
  ) {
    items { id data }
    total
  }
}

# Filtrer les exemplaires dont le livre appartient au genre 'Polar' (2
 ↪  niveaux)
query {
  records(
    spaceId: "exemplaires-id"
    filter: { formula: "@livre_id.genre_id.libelle == 'Polar'", language:
 ↪  "moonscript" }
  ) {
    items { id data }
    total
  }
}
```

Les formules FK fonctionnent également dans les **requêtes dynamiques** générées pour chaque espace :

```
query {
  livres(filter: { formula: "@genre_id.libelle == 'Roman'", language:
 ↪  "moonscript" }) {
    items { titre }
  }
}
```

> **Note** : La résolution FK utilise d'abord un accès direct par clé primaire (efficace), avec repli sur un full scan pour les références non-PK. Sur des espaces de grande taille, préférer les filtres par champ classiques lorsque c'est possible.

API GraphQL

```
# Lire les enregistrements
query {
  records(spaceId: "...", limit: 100) {
    items { id data } total
  }
}

# Lire un seul enregistrement
query { record(spaceId: "...", id: "42") { id data } }
```

```
# Insérer un auteur
mutation {
  insertRecord(
    spaceId: "...",
    data: { nom: "Hugo", prenom: "Victor", particule: "" }
  ) { id data }
}

# Insérer un livre
mutation {
  insertRecord(
    spaceId: "...",
    data: { titre: "Les Misérables", auteur_id: 1, genre_id: 2, annee: 1862,
↪   isbn: "978-2-07-040850-4" }
  ) { id data }
}

# Insérer un exemplaire
mutation {
  insertRecord(
    spaceId: "...",
    data: { livre_id: 1, etat: "bon", disponible: true }
  ) { id data }
}

# Modifier
mutation {
  updateRecord(spaceId: "...", id: "42", data: { nom: "Martin" }) {
    id data
  }
}

# Supprimer
mutation { deleteRecord(spaceId: "...", id: "42") }
```

---

5. Vues classiques

Les **vues classiques** sont des configurations nommées d'affichage d'un espace (type grille, formulaire ou galerie). Actuellement le type `grid` est pleinement pris en charge.

API GraphQL

```
# Créer une vue grille sur le catalogue des livres
```

```
mutation {
  createView(
    spaceId: "...",
    input: { name: "Catalogue", viewType: grid }
  ) { id }
}

# Modifier
mutation {
  updateView(id: "...", input: { name: "Nouveau nom" }) { id name }
}

# Supprimer
mutation { deleteView(id: "...") }
```

---

6. Vues personnalisées (YAML)

Les **vues personnalisées** sont des tableaux de bord multi-espaces déclarés en YAML.

Éditeur

Cliquer « **Éditer** » sur une vue ouvre un modal plein écran composé de :

- Un éditeur **CodeMirror** (YAML, thème monokai) pour saisir directement le YAML.
- Un **diagramme ERD interactif** à droite : cliquer sur un champ génère automatiquement le YAML correspondant dans l'éditeur (widget, colonnes, `depends_on` si FK détectée).

Chaque boîte ERD dispose d'un pseudo-champ * (première rangée) qui ajoute l'espace sans restriction de colonnes (pas de clé `columns` dans le YAML). Les champs sont classés **par ordre alphabétique**. Les auto-relations (ex. `parent_id → id`) sont représentées par une boucle sur le côté droit de la boîte.

Structure de base

```yaml
layout:
  direction: vertical    # ou horizontal
  children:
    - widget: ...        # widget feuille (grille de données)
    - layout: ...        # zone imbriquée (récursif)
```

Widget

```yaml
widget:
  id: identifiant_unique   # requis, unique dans la vue
```

```
  title: "Titre affiché"   # optionnel
  space: nom_espace        # nom de l'espace Tarantool
  columns: [champ1, champ2]  # colonnes affichées (défaut : toutes)
  factor: 2                  # poids dans la zone parente (défaut : 1)
  depends_on:                # filtrage dépendant (optionnel)
    widget: id_widget_parent
    field: champ_jointure    # champ dans CET espace
    from_field: id           # champ dans le widget parent (défaut : id)
```

Dépendances (filtrage inter-widgets)

Quand `depends_on` est défini, le widget n'affiche que les enregistrements dont le champ `field` correspond à la valeur du champ `from_field` de la ligne sélectionnée dans le widget parent.

Facteurs de proportion

Le `factor` indique la place relative prise par un enfant dans sa zone. Exemple : si un widget A a `factor: 2` et un widget B `factor: 1`, A occupe 2/3 de l'espace.

Colonnes personnalisées

```
columns: [prenom, nom, email]   # seuls ces champs, dans cet ordre
```

Si omis, tous les champs de l'espace sont affichés.

Exemple complet

Vue **Gestion de la bibliothèque** : catalogue des livres → exemplaires → emprunts en cours, avec un widget agrégat des emprunts par genre.

```
layout:
  direction: vertical
  children:
    - widget:
        id: livres
        title: Catalogue des livres
        space: livres
        columns: [titre, cote_auto, auteur_id, genre_id, isbn, annee]
    - layout:
        direction: horizontal
        children:
          - widget:
              id: exemplaires
              title: Exemplaires
              space: exemplaires
              depends_on:
```

```yaml
            widget: livres
            field: livre_id
            from_field: id
          columns: [etat, disponible]
          factor: 2
        - widget:
          id: emprunts
          title: Emprunts en cours
          space: emprunts
          depends_on:
            widget: exemplaires
            field: exemplaire_id
            from_field: id
          columns: [personne_id, date_emprunt, date_retour]
          factor: 3
```

Widget agrégat

Un widget de type `aggregate` affiche un tableau de synthèse en lecture seule, calculé côté serveur par itération sur l'espace. Il est équivalent à un `GROUP BY` SQL.

```yaml
widget:
  type: aggregate
  title: "Emprunts par genre"
  space: emprunts
  groupBy: [genre_id]
  aggregate:
    - fn: count
      as: nb_emprunts
```

L'alias `as` est optionnel : s'il est absent, le nom de colonne généré est `fn_field` (ex. `avg_annee`) ou `count` pour `COUNT(*)`.

Colonnes calculées Il est possible d'ajouter des colonnes calculées côté client via la clé `computed`. Chaque entrée fournit un alias (`as`) et une expression JavaScript (`expr`) évaluée sur chaque ligne agrégée. L'objet `row` contient tous les champs `groupBy` et les alias `aggregate`.

```yaml
widget:
  type: aggregate
  title: "Emprunts par genre"
  space: emprunts
  groupBy: [genre_id]
  aggregate:
    - fn: count
      as: nb_emprunts
  computed:
```

```yaml
    - as: label
      expr: "row.genre_id + ' (' + row.nb_emprunts + ')'"
    - as: statut
      expr: "row.nb_emprunts > 10 ? 'Actif' : 'Peu emprunté'"
```

Note : les expressions `computed` sont du JavaScript pur (pas du MoonScript) ; elles s'exécutent dans le navigateur, après réception des données agrégées.

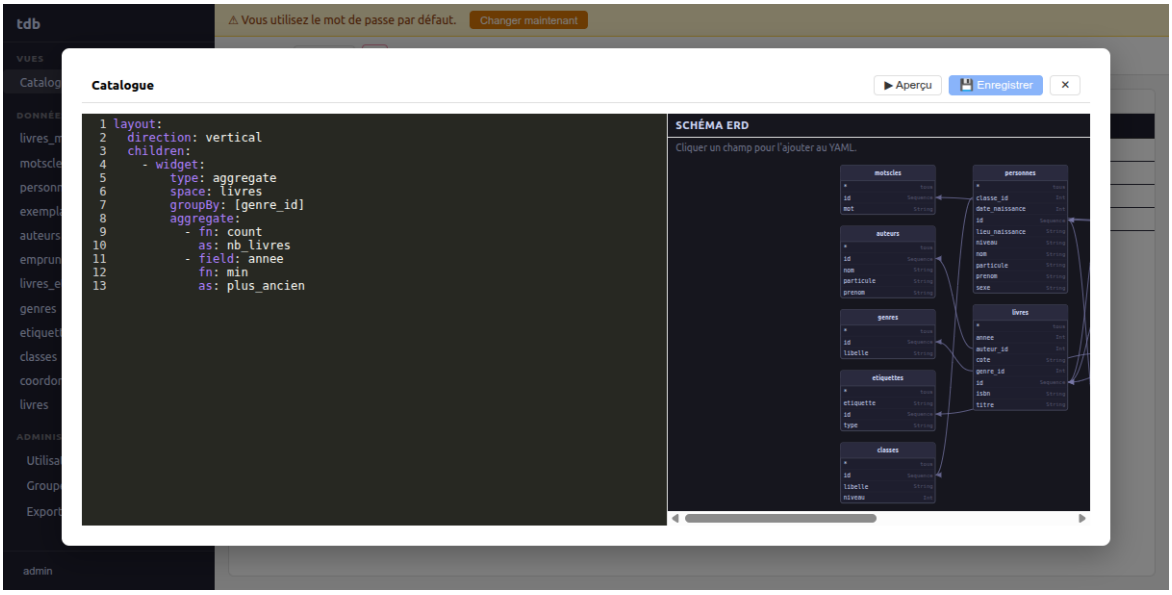

FIGURE 1 – Widget agrégat

API GraphQL

```graphql
# Lister
query { customViews { id name yaml } }

# Créer
mutation {
  createCustomView(input: {
    name: "Tableau de bord",
    yaml: "layout:\n  direction: vertical\n  children: []\n"
  }) { id }
}

# Modifier
mutation {
  updateCustomView(id: "...", input: { yaml: "..." }) { id yaml }
}

# Supprimer
mutation { deleteCustomView(id: "...") }
```

```
# Requête d'agrégation directe : emprunts par genre
query {
  aggregateSpace(
    spaceName: "emprunts",
    groupBy: ["genre_id"],
    aggregate: [
      { fn: "count", as: "nb_emprunts" }
    ]
  )
}
```

---

7. Authentification

TGui utilise des **tokens Bearer** (SHA-256 + sel, stockés en session Tarantool). Le token est persisté dans `localStorage` et envoyé automatiquement par le client.

Flux de connexion

```
mutation {
  login(username: "alice", password: "secret") {
    token
    user { id username email groups { id name } }
  }
}
```

Le token est ensuite envoyé dans l'en-tête HTTP :

```
Authorization: Bearer <token>
```

Déconnexion

```
mutation { logout }
```

Utilisateur courant

```
query { me { id username email groups { id name } } }
```

Changer son propre mot de passe

```
mutation {
  changePassword(currentPassword: "ancien", newPassword: "nouveau")
}
```

---

8. Utilisateurs et groupes

Ces opérations nécessitent d'être **membre du groupe `admin`**.

Utilisateurs

```
# Lister
query { users { id username email groups { id name } } }

# Créer (admin uniquement)
mutation {
  createUser(input: {
    username: "bob",
    email: "bob@ex.com",
    password: "secret"
  }) { id }
}

# Forcer un changement de mot de passe (admin uniquement)
mutation {
  adminSetPassword(userId: "...", newPassword: "nouveau")
}
```

Groupes

```
# Lister (avec membres et permissions)
query {
  groups {
    id name description
    members { id username }
    permissions { id resourceType resourceId level }
  }
}

# Créer
mutation {
  createGroup(input: { name: "editeurs", description: "..." }) { id }
}

# Supprimer
mutation { deleteGroup(id: "...") }

# Ajouter un membre
```

```
mutation { addMember(userId: "...", groupId: "...") }

# Retirer un membre
mutation { removeMember(userId: "...", groupId: "...") }
```

---

9. Droits (permissions)

Les permissions s'accordent à un **groupe** sur une ressource.

Niveau	Description
`read`	Lecture seule
`write`	Lecture + écriture
`admin`	Contrôle total de la ressource

```
# Accorder un droit
mutation {
  grant(groupId: "...", input: {
    resourceType: "space",
    resourceId: "...",
    level: write
  }) { id }
}

# Révoquer
mutation { revoke(permissionId: "...") }
```

---

10. Export et import de snapshots

La section **Export / Import** du panneau Administration permet de sauvegarder l'intégralité d'une application TGui (schéma, vues personnalisées, groupes, permissions, et optionnellement les données) dans un fichier `.tdb.yaml`, puis de le restaurer sur une autre instance.

Export

Deux modes d'export sont disponibles :

- **Structure seule** (`exportSnapshot(includeData: false)`) : exporte le schéma complet (espaces, champs, relations, vues personnalisées, groupes, permissions) sans les données. Utile pour partager un modèle d'application.

— **Structure + données** (`exportSnapshot(includeData: true)`) : exporte également toutes les lignes de chaque espace. Utile pour une sauvegarde complète ou une migration.

Le fichier produit est du YAML lisible :

```yaml
version: "1"
exported_at: "2026-03-08T12:00:00"
schema:
  spaces:
    - name: auteurs
      fields:
        - name: particule
          fieldType: String
          notNull: false
        - name: nom
          fieldType: String
          notNull: true
        - name: prenom
          fieldType: String
          notNull: false
        - name: nom_complet
          fieldType: String
          formula: "=> parts = {}\ntable.insert(parts, @prenom) if @prenom and
↪ @prenom != \"\"\ntable.insert(parts, @particule) if @particule and
↪ @particule != \"\"\ntable.insert(parts, @nom) if @nom and @nom !=
↪ \"\"\ntable.concat(parts, \" \")"
          language: moonscript
    - name: genres
      fields:
        - name: libelle
          fieldType: String
          notNull: true
    - name: livres
      fields:
        - name: titre
          fieldType: String
          notNull: true
        - name: auteur_id
          fieldType: Relation
          notNull: false
        - name: genre_id
          fieldType: Relation
          notNull: false
        - name: cote
          fieldType: String
          notNull: false
        - name: isbn
```

```yaml
          fieldType: String
          notNull: false
        - name: annee
          fieldType: Int
          notNull: false
        - name: cote_auto
          fieldType: String
          formula: "=> prefix = (@titre or \"\")\\upper!\\sub(1,
↪  3)\\gsub(\"[^%A]\", \"\")\nannee  = if @annee then tostring(@annee) else
↪  \"????\"\n\"#{prefix}-#{annee}\""
          triggerFields: [titre, annee]
          language: moonscript
      views:
        - name: Catalogue
          viewType: Grid
  relations:
    - fromSpace: livres
      fromField: auteur_id
      toSpace: auteurs
      toField: id
    - fromSpace: livres
      fromField: genre_id
      toSpace: genres
      toField: id
    - fromSpace: exemplaires
      fromField: livre_id
      toSpace: livres
      toField: id
  custom_views:
    - name: Gestion de la bibliothèque
      yaml: |
        layout: …
  groups:
    - name: admin
      members: [admin]
      permissions:
        - resourceType: space
          level: admin
data:                    # absent si export structure seulement
  auteurs:
    - {nom: Hugo, prenom: Victor}
  genres:
    - {libelle: Roman}
  livres:
    - {titre: "Les Misérables", auteur_id: 1, genre_id: 1, annee: 1862}
```

Les identifiants internes sont exclus ; la correspondance lors de l'import se fait par **nom**.

Import

1. Cliquer sur **Choisir un fichier** et sélectionner un fichier `.tdb.yaml`.
2. TGui calcule automatiquement un **diff** entre le schéma du fichier et le schéma courant, et l'affiche avec un code couleur :
   — vert **Créer** : l'élément sera ajouté.
   — rouge **Supprimer** : l'élément sera retiré (mode Remplacement uniquement).
   — orange **Modifier** : le type d'un champ a changé.
3. Choisir le **mode** d'import et confirmer.

Modes d'import

Mode	Comportement
Fusion (`merge`)	Crée les espaces, champs, relations, vues et groupes manquants. Les éléments déjà présents (même nom) sont laissés intacts.
Remplacement (`replace`)	Supprime d'abord tous les espaces et groupes existants, puis recrée tout depuis le snapshot. Attention : toutes les données existantes sont perdues.

Le résultat de l'opération indique le nombre d'éléments créés, ignorés et les erreurs éventuelles.

API GraphQL correspondante

```
# Export
query Export($includeData: Boolean!) {
  exportSnapshot(includeData: $includeData)
}

# Prévisualisation du diff
query Diff($yaml: String!) {
  diffSnapshot(yaml: $yaml) {
    spacesToCreate
    spacesToDelete
    customViewsToCreate
    customViewsToUpdate
  }
}

# Import
mutation Import($yaml: String!, $mode: ImportMode!) {
```

```
  importSnapshot(yaml: $yaml, mode: $mode) {
    ok
    created
    skipped
    errors
  }
}
```

---

11. API GraphQL complète

Queries

Query	Description
`spaces`	Liste tous les espaces
`space(id)`	Détail d'un espace
`views(spaceId)`	Vues classiques d'un espace
`view(id)`	Détail d'une vue classique
`customViews`	Toutes les vues personnalisées
`customView(id)`	Détail d'une vue personnalisée
`records(spaceId, filter, limit, offset)`	Enregistrements paginés et filtrés
`record(spaceId, id)`	Un enregistrement
`relations(spaceId)`	Relations d'un espace
`me`	Utilisateur connecté
`users`	Tous les utilisateurs (admin)
`user(id)`	Détail d'un utilisateur (admin)
`groups`	Tous les groupes (admin)
`group(id)`	Détail d'un groupe (admin)
`exportSnapshot(includeData)`	Export YAML de l'application (admin)
`diffSnapshot(yaml)`	Prévisualisation du diff avant import (admin)
`aggregateSpace(spaceName, groupBy, aggregate)`	Requête d'agrégation sur un espace

Mutations

Mutation	Description
`createSpace` / `updateSpace` / `deleteSpace`	CRUD espaces
`addField` / `updateField` / `removeField` / `reorderFields`	CRUD champs
`createView` / `updateView` / `deleteView`	CRUD vues classiques
`createRelation` / `updateRelation` / `deleteRelation`	CRUD relations
`createCustomView` / `updateCustomView` / `deleteCustomView`	CRUD vues YAML
`insertRecord` / `updateRecord` / `deleteRecord`	CRUD enregistrement unitaire
`insertRecords` / `updateRecords` / `deleteRecords`	Opérations batch sur enregistrements
`addFields`	Ajout multiple de champs
`login` / `logout`	Authentification
`createUser`	Créer un utilisateur (admin)
`changePassword`	Changer son propre mot de passe
`adminSetPassword`	Forcer le mot de passe d'un utilisateur (admin)
`createGroup` / `deleteGroup`	CRUD groupes (admin)
`addMember` / `removeMember`	Gestion des membres (admin)
`grant` / `revoke`	Gestion des permissions (admin)
`importSnapshot(yaml, mode)`	Import d'un snapshot YAML (admin)

Architecture technique

```
┌──────────────────────────────────────────────────────┐
│  Navigateur                                          │
│  CoffeeScript → JS  (app, auth, spaces, data_view,  │
│                       custom_view, graphql_client)   │
│  tui-grid (grille)  │  jsyaml (parsing YAML)         │
└──────────────────────┴──────────────────────────────┘
                       │ HTTP + GraphQL (JSON)
┌──────────────────────▼──────────────────────────────┐
│  Tarantool 3.x                                       │
```

```
│  MoonScript → Lua                                    │
│  ├── http_server (tarantool/http)                     │
│  ├── graphql engine (maison, pur Lua)                 │
│  ├── resolvers/  (schema, data, auth, custom_view)    │
│  ├── core/       (auth, permissions, formula)         │
│  └── _tdb_*      (spaces internes Tarantool)          │
└──────────────────────────────────────────────────────┘
```

Composant	Technologie
Backend	MoonScript compilé en Lua, exécuté par Tarantool
Frontend	CoffeeScript compilé en JS, servi statiquement
Base de données	Tarantool (espaces internes `_tdb_*`)
API	GraphQL maison (pas de dépendance npm/lua tierce)
Authentification	Bearer token, SHA-256 + sel, sessions en mémoire Tarantool
Tests	Runner CoffeeScript maison (pas de Jest/Mocha)